Dinamikus Programozás 2.

DP intervallumokon és részhalmazokon, rekurzió memorizálással

Szerző: Nikházy László
Előadó: Busa Máté

2025. november 28.

A dinamikus programozás stratégiája

A dinamikus programozás, mint probléma-megoldási stratégia az alábbi öt lépés végrehajtását jelenti.

1. **Az [optimális] megoldás szerkezetének elemzése.**

2. **Részproblémákra és összetevőkre bontás** úgy, hogy:

a) az összetevőktől való függés körmentes legyen,

b) minden részprobléma [optimális] megoldása kifejezhető legyen (rekurzívan) az összetevők [optimális] megoldásaival.

3. **Részproblémák [optimális] megoldásának rekurzív kifejezése** az összetevők [optimális] megoldásaiból. Másnéven: átmenet.

4. **Részproblémák [optimális] megoldásának kiszámítása alulról-felfelé:**

a) a kiszámítási sorrend meghatározása (minden részprobléma összetevői előbb szerepeljenek),

b) a részproblémák kiszámítása táblázat-kitöltéssel.

5. **Egy [optimális] megoldás előállítása** a 4. lépésben kiszámított (és tárolt) információkból. Másnéven: rekonstrukció.

Dinamikus programozás - Részproblémák

A DP megoldások kulcsa a megfelelő részproblémák megtalálása. Erre általános recept nincs, de néhány tipikus séma van, ami sokszor előfordul:

- egy paraméter minden lehetséges értékét vesszük (pl. hátizsák méret)
- egy sorozat minden kezdőszelete (pl. első `i` elemre DP)
- egy keresési tér minden pozíciója (pl. négyzetrács mezői, gráf csúcsai)
- **egy sorozat minden `(i, j)` intervalluma**
- **az elemek minden részhalmaza**

DP intervallumokon

(Range DP / Substring DP)

Leghosszabb tükörszó hossza

Egy szót *tükörszónak* (palindromnak) nevezünk, ha előről és hátulról olvasva megegyezik. Egy szóban tükörszó lehet úgy is, hogy betűket törlünk belőle (tehát részsorozat).

Feladat: Adott egy szó `S`. Határozd meg az `S`-ben található leghosszabb tükörszó **hosszát** (részsorozatként).

Bemenet: Egy sor — a szó `S` (hossza ≤ 1000).

Kimenet: Egy egész szám — a leghosszabb tükörszó hossza.

Példa:

Bemenet:
`abbakabadara`

Kimenet:
7

Leghosszabb tükörszó hossza – Szerkezet és részproblémák

Szerkezet: Ha egy optimális tükörszó `S[i...j]`-ben kezdődik és végződik, akkor belső része is optimális a megfelelő intervallumra. Ez alkalmas intervallum-DP-re.

Részprobléma:
`dp[i][j]` = a leghosszabb palindrom részsorozat hossza az `S[i..j]` substringben ($1 \leq \mathtt{i} \leq \mathtt{j} \leq \mathtt{n}$).

Leghosszabb tükörszó hossza – Szerkezet és részproblémák

Szerkezet: Ha egy optimális tükörszó `S[i...j]`-ben kezdődik és végződik, akkor belső része is optimális a megfelelő intervallumra. Ez alkalmas intervallum-DP-re.

Részprobléma:
`dp[i][j]` = a leghosszabb palindrom részsorozat hossza az `S[i..j]` substringben ($1 \leq \mathtt{i} \leq \mathtt{j} \leq \mathtt{n}$).

Alapesetek:
`dp[i][i]` = 1 (egy karakter önmagában palindrom),
ha `i` > `j` akkor `dp[i][j]` = 0 (üres intervallum).

Átmenet:

$$\mathtt{dp[i][j]} = \begin{cases} \mathtt{dp[i+1][j-1]} + 2, & \text{ha } \mathtt{S[i]} == \mathtt{S[j]} \\ \mathtt{max(dp[i+1][j],\ dp[i][j-1])}, & \text{ha } \mathtt{S[i]} \neq \mathtt{S[j]}. \end{cases}$$

(Értelmezés: ha a két vége egyezik, hozzácsatolhatjuk őket; különben egyik végét elhagyva keressük a jobb megoldást.)

Leghosszabb tükörszó hossza – Számítási sorrend, rekonstrukció, komplexitás

Számítási sorrend:
Kitöltés intervallum-hossz szerint: először $\ell = 1$ (átló), majd $\ell = 2, \ldots, \mathtt{n}$. Minden ℓ-re végigmegyünk $\mathtt{i} = 1 \ldots \mathtt{n} - \ell + 1$, $\mathtt{j} = \mathtt{i} + \ell - 1$.

Rekonstrukció, ha kellene:

- Induljunk az $\mathtt{i} = 1$, $\mathtt{j} = \mathtt{n}$ állapotból.
- Amíg $\mathtt{i} < \mathtt{j}$:
  - ha `S[i] == S[j]`: adjuk hozzá a karaktert a megoldáshoz; `i++, j--`.
  - különben ha $\mathtt{dp[i+1][j]} \geq \mathtt{dp[i][j-1]}$ akkor `i++`, különben `j--`.
- A végén meglesz a palindrom egyik fele, illetve ha `i == j` akkor még `S[i]`-t hozzá kell venni középső karakternek.

Komplexitás: idő és memória: $\mathtt{O(n^2)}$

Leghosszabb tükörszó hossza – Kódvázlat

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      string s; cin >> s;
5      int n = s.size();
6      s = "␣" + s;
7      vector<vector<int>> dp(n+2, vector<int>(n+2, 0));
8      for (int i = 1; i <= n; ++i) dp[i][i] = 1;
9      for (int len = 2; len <= n; ++len) {
10         for (int i = 1; i + len - 1 <= n; ++i) {
11             int j = i + len - 1;
12             if (s[i] == s[j]) {
13                 dp[i][j] = dp[i+1][j-1] + 2;
14             } else {
15                 dp[i][j] = max(dp[i+1][j], dp[i][j-1]);
16             }
17         }
18     }
19     cout << dp[1][n] << "\n";
20 }
```

Removal Game (CSES) - Feladat

Adott egy n hosszúságú sorozat $x_1, \ldots, x_n$. Két játékos felváltva lép: egy lépésben az aktuális játékos törölheti a sorozat első vagy utolsó elemét, és hozzáadja azt a saját pontszámához. Mindketten optimálisan játszanak (a saját pontszámukat maximalizálják). Határozd meg, hogy a kezdő játékos legfeljebb hány pontot érhet el, ha mindkét játékos optimálisan játszik.

Bemenet: Első sor: n. Második sor: $x_1\ x_2\ \ldots\ x_n$.

Kimenet: A kezdő játékos maximális elérhető pontszáma.

Korlátok: $1 \leq n \leq 5000, \quad -10^9 \leq x_i \leq 10^9$

Példa:

Bemenet:

```
4
4 5 1 3
```

Kimenet:

```
8
```

Removal Game – Szerkezet és részproblémák

Szerkezet: A játék egy aktuális állapota jellemezhető a két szélső meglévő szám indexével: `(i, j)` intervallum (még a soron következő játékos is meghatározott ezáltal). Bármilyen közbülső állapot ugyanúgy viselkedik, mint a kezdőállapot.
Ha az `(i, j)` intervallumot vizsgáljuk, akkor a kezdőjátékos választhat: x_i-t veszi el, és a másik játékos az `(i + 1, j)` helyzetből kezdhet, vagy x_j-t veszi el, és a másik játékos az `(i, j − 1)` helyzetből kezdhet.

Részprobléma: `dp[i][j]` = a két játékos pontszámának *különbsége* (kezdő – második), ha az `(i, j)` intervallumban játszunk.

Miért pont különbség? Mert így mindig a következő játékos szemszögéből számolunk majd.

Removal Game – Rekurzív összefüggés

Átmenet:
Ha a kezdő játékos x_i-t veszi el:

$$\text{eredmény} = x_i - dp[i+1][j]$$

Mert a következő játékos a $[i+1, j]$ intervallumban lesz *kezdő*, és $dp[i+1][j]$ a következő játékos különbsége.

Ha a kezdő játékos x_j-t veszi el:

$$\text{eredmény} = x_j - dp[i][j-1]$$

Képlet: $dp[i][j] = \max\left(x_i - dp[i+1][j],\ \ x_j - dp[i][j-1]\right)$

Alapeset: $dp[i][i] = x_i$

Végső válasz:

$$\text{első játékos pontszáma} = \frac{\sum x_i + dp[1][n]}{2}$$

Kiszámítási sorrend: átlósan (intervallum hossza szerint), vagy pedig $i = n \ldots 1,\ j = 1 \ldots i$

Időkomplexitás: $O(n^2)$ **Memória:** $O(n^2)$

Removal Game – Kód

Kód:

```
1   #include <bits/stdc++.h>
2   using namespace std;
3
4   long long dp[5005][5005];
5
6   int main() {
7       int n;
8       cin >> n;
9       vector<long long> x(n+1);
10      long long S = 0;
11      for (int i = 1; i <= n; ++i) {
12          cin >> x[i];
13          S += x[i];
14          dp[i][i] = x[i];
15      }
16
17      for (int len = 2; len <= n; ++len) {
18          for (int i = 1; i+len-1 <= n; ++i) {
19              int j = i + len - 1;
20              dp[i][j] = max(x[i] - dp[i+1][j],
21                             x[j] - dp[i][j-1]);
22          }
23      }
24
25      cout << (S + dp[1][n]) / 2 << "\n";
26  }
```

Clear the String (Codeforces) - Feladat

Adott egy `n` hosszúságú kisbetűs `s` string. Egy művelettel kitörölhetsz belőle egy olyan substringet, amelyben **minden karakter egyenlő**. (A törlés után a maradék karakterek összeolvadnak.) Határozd meg a teljes string törléséhez szükséges **minimális** műveletszámot!

Bemenet:
Első sor: `n` ($1 \leq n \leq 500$).
Második sor: az `s` string (hossza `n`).

Kimenet: A minimális törlések száma.

Példák:

Bemenet:
5
ababa

Kimenet:
3

Bemenet:
8
abcddcba

Kimenet:
4

Clear the String — Részprobléma és átmenet

Részprobléma: `dp[l][r]` = minimális műveletszám, amivel a `s[l..r]` teljesen kitörölhető ($1 \leq \texttt{l} \leq \texttt{r} \leq \texttt{n}$).

Alapeset: `dp[i][i]` = 1 (egy karaktert egy lépésben törlünk).

Átmenet:

Clear the String — Részprobléma és átmenet

Részprobléma: `dp[l][r]` = minimális műveletszám, amivel a `s[l..r]` teljesen kitörölhető ($1 \leq l \leq r \leq n$).

Alapeset: `dp[i][i]` = 1 (egy karaktert egy lépésben törlünk).

Átmenet:

- Kezdetben legyen `dp[l][r]` = 1 + `dp[l][r-1]` (töröljük `s[r]` külön).
- Ha létezik `k` ($l \leq k < r$) olyan, hogy `s[k]` = `s[r]`, akkor összekapcsolhatjuk `s[r]` törlését azzal a részlettel: `dp[l][r] = min(dp[l][r], dp[l][k] + dp[k+1][r-1])`. (Az értelmezés: először kitöröljük `s[k+1..r-1]`, így `s[k]` és `s[r]` szomszédossá válik, és egy lépéssel együtt törölhetők.)

Számítási sorrend: növekvő intervallumhossz `len` = `1..n`, és minden `l`-re `r` = `l` + `len` − 1.

Komplexitás: $O(n^3)$ idő, $O(n^2)$ memória;

Clear the String — Kódvázlat

```
1   int n; cin >> n;
2   string s; cin >> s;
3   vector<vector<int>> dp(n, vector<int>(n, 0));
4   for (int i = 0; i < n; ++i) dp[i][i] = 1;
5   for (int len = 2; len <= n; ++len) {
6       for (int l = 0; l + len - 1 < n; ++l) {
7           int r = l + len - 1;
8           dp[l][r] = 1 + dp[l][r-1];
9           for (int k = l; k < r; ++k) {
10              if (s[k] == s[r]) {
11                  dp[l][r] = min(dp[l][r], dp[l][k] + dp[k+1][r-1]);
12              }
13          }
14      }
15  }
16  cout << dp[0][n-1] << "\n";
```

DP részhalmazokon

(Bitmask DP / Subset DP)

Matching (AtCoder) - Feladat

Adott $\mathtt{N}$ férfi és $\mathtt{N}$ nő (sorszámuk $1, \ldots, \mathtt{N}$). Minden párra meg van adva egy $a_{i,j} \in \{0, 1\}$ érték: ha $a_{i,j} = 1$, akkor a $\mathtt{i}$-edik férfi és a $\mathtt{j}$-edik nő kompatibilis (összepárosítható), különben nem. Hány különböző módon lehet párokat létrehozni úgy, hogy minden férfi és nő pontosan egy párban szerepeljen, és minden pár kompatibilis legyen? A választ modulo $10^9 + 7$ kell megadni.

Korlátok: $1 \leq \mathtt{N} \leq 21$.

Példa:

Bemenet:

```
3
0 1 1
1 0 1
1 1 1
```

Kimenet:

```
3
```

A három lehetséges párosítás:
(1,2),(2,1),(3,3);
(1,2),(2,3),(3,1);
(1,3),(2,1),(3,2).

Matching — Részproblémák

Szerkezet: Ha az első `k` férfit párosítjuk, akkor a nők egy `k`-elemű részhalmaza lesz foglalt.

Részprobléma: `dp[k][s]` = hányféleképpen lehet az **első** `k` **férfit** összepárosítani a **nők** `s` **részhalmazával**.

Reprezentáció: A részhalmazokat *bitmaszk*ként kezeljük, a $0 \ldots 2^n - 1$ számok mindegyike egy részhalmazt kódol kettes számrendszerben, a `j`-edik bit jelzi, hogy a `j`-edik nő benne van-e.

Megjegyzés: Csak azok a maszkok érdekesek, amelyekben `k` 1-es bit van (`popcount(s)` = `k`), különben `dp[k][s]` = 0.

Matching — Átmenet

Tekintsük most a k-adik férfit. Ha s tartalmazza a j-edik nőt és $a_{k,j} = 1$ (kompatibilisek), akkor a párosítások száma, ahol az első k férfi pont a s-beli nőket kapja, kialakulhat úgy, hogy a k-adik férfi a j-edik nőt kapja, a korábbi $k-1$ férfi pedig a $s \setminus \{j\}$ részhalmazt fedi le.

Átmenet:

$$dp[k][s] = \sum_{\substack{j \in s, \\ a_{k,j}=1}} dp[k-1][s \setminus \{j\}].$$

Kezdeti érték: $dp[0][0] = 1$, minden más $dp[0][s \neq 0] = 0$.

Komplexitás: $O(2^n \cdot n)$ — mert minden s-re végigpróbáljuk a j-ket, és minden s részhalmaz csak egyszer kerül sorra, a megfelelő k-nál. Valójában a dp első dimenziója (k) elhagyható lenne, hiszen s bitjeinek száma implicit kódolja ezt.

Matching — Kód

```
1   const int MOD = 1000000007;
2   int n; cin >> n;
3   vector<vector<int>> a(n, vector<int>(n));
4   for (int i = 0; i < n; i++) {
5       for (int j = 0; j < n; j++) {
6           cin >> a[i][j];
7       }
8   }
9   const int ALL = 1 << n;
10  vector<vector<int>> dp(n + 1, vector<int>(ALL));
11  dp[0][0] = 1;
12
13  for (int k = 1; k <= n; k++) {
14      for (int mask = 0; mask < ALL; mask++) {
15          if (__builtin_popcount(mask) != k) continue;
16          for (int j = 0; j < n; j++) {
17              if ((mask >> j & 1) && a[k - 1][j]) {
18                  dp[k][mask] += dp[k - 1][mask ^ (1 << j)];
19                  if (dp[k][mask] >= MOD) dp[k][mask] -= MOD;
20              }
21          }
22      }
23  }
24
25  cout << dp[n][ALL - 1] << "\n";
```

Vásárlások (Válogató 2018) - Feladat

Ismert $\mathtt{K}$ termék napi ára $\mathtt{N}$ napon keresztül. Az $\mathtt{N}$ nap alatt mind a $\mathtt{K}$ terméket meg kell vásárolni, de naponta legfeljebb 1 terméket vehetsz. Adjunk meg vásárlási napokat (melyik terméket melyik napon vesszük), hogy az összköltség minimális legyen.

Bemenet: Első sor: $\mathtt{K}\ \mathtt{N}$ $\quad (1 \leq \mathtt{K} < 10, \mathtt{K} \leq \mathtt{N} \leq 100)$.
Következő $\mathtt{K}$ sor: az egyes termékek $\mathtt{N}$ napi ára (egész).

Kimenet: Első sor: a minimális teljes költség $\mathtt{M}$.
Második sor: $\mathtt{K}$ szám — az $\mathtt{i}$-edik szám azt a napot adja meg, amikor az $\mathtt{i}$-edik terméket megvesszük. (Több megoldás esetén bármelyik elfogadható.)

Példa — Bemenet:

3 8
6 9 5 **2** 8 9 1 6
8 **2** 6 2 7 5 7 2
5 3 9 7 3 5 **1** 4

Példa — Kimenet:

5
4 2 7

Magyarázat: a pirossal jelölt számok a fenti bemenetben az optimális választások (1. termék: 4. nap $\rightarrow$ 2, 2. termék: 2. nap $\rightarrow$ 2, 3. termék: 7. nap $\rightarrow$ 1), összesen $2 + 2 + 1 = 5$.

Vásárlások — Megoldás

Részproblémák: Legyen `dp[i][s]` = a minimális költség, hogy az első `i` napon a termékek `s` részhalmazát megvegyük. (`s`-et bitmaszkkal reprezentáljuk.)

Alapesetek: `dp[0][0]` $= 0$, minden más `dp[0][s` $\neq 0$`]` $= \infty$.

Átmenet:

$$\texttt{dp[i][s]} = \min\left(\texttt{dp[i-1][s]},\ \min_{j \notin s} \texttt{dp[i-1][s}\setminus\texttt{\{j\}]} + \text{price}\texttt{[i][j]}\right)$$

Értelmezés:

- Nem veszünk semmit ezen a napon: `dp[i][s]` $=$ `dp[i` -1`][s]`
- Megvesszük a `j` terméket: `dp[i][s]` $=$ `dp[i` -1`][s` $\setminus$ `{j}]` $+$ ár

Komplexitás: $\texttt{O(N}\cdot 2^{\texttt{K}}\cdot\texttt{K)}$, ami elég gyors, ha $\texttt{K} < 10$, $\texttt{N} \leq 100$.

Vásárlások — Kód (DP számítás)

```
1  int K, N; cin >> K >> N;
2  const int P = 1 << K;
3  const int INF = 1e9;
4  vector<vector<int>> price(N+1, vector<int>(K));
5  for (int j = 0; j < K; ++j)
6      for (int i = 1; i <= N; ++i)
7          cin >> price[i][j];
8
9  vector<vector<int>> dp(N+1, vector<int>(P, INF));
10 vector<vector<int>> bought(N+1, vector<int>(P, -1));
11 dp[0][0] = 0;
12 for (int i = 1; i <= N; ++i) {
13     for (int mask = 0; mask < P; ++mask) {
14         dp[i][mask] = dp[i-1][mask];
15         for (int j = 0; j < K; ++j) if (mask>>j & 1) {
16             int pmask = mask ^ (1<<j);
17             if (dp[i-1][pmask] + price[i][j] < dp[i][mask]) {
18                 dp[i][mask] = dp[i-1][pmask] + price[i][j];
19                 bought[i][mask] = j;
20             }
21         }
22     }
23 }
24 cout << dp[N][P-1] << "\n";
```

Vásárlások — Kód (Rekonstrukció)

```
1   vector<int> when(K, 0);
2   int mask = P-1;
3   for (int i = N; i >= 1; --i) {
4       int j = bought[i][mask];
5       if (j != -1) {
6           when[j] = i;
7           mask ^= (1<<j);
8       }
9   }
10
11  for (int j = 0; j < K; ++j) {
12      cout << when[j] << "␣";
13  }
14  cout << "\n";
```

Hamiltonian Flights (CSES) - Feladat

Adott n város és m irányított járat. Szeretnél Syrjäläből (1-es város) Lehmäläba (n) utazni úgy, hogy minden várost pontosan egyszer látogatsz meg. Hány különböző útvonal lehetséges? (Az eredményt modulo $10^9 + 7$ írd ki.)

Bemenet: Első sor: n m. Következő m sor: két egész a b — van irányított $a \to b$ járat.

Kimenet: Egy egész — az útvonalak száma modulo $10^9 + 7$.

Korlátok: $2 \leq n \leq 20$, $1 \leq m \leq n^2$.

Példa bemenet:

```
4 6
1 2
1 3
2 3
3 2
2 4
3 4
```

Kimenet:

```
2
```

Hamiltonian Flights — Megoldás

Szerkezet: Az aktuális állapot két adattal írható le:

- melyik csúcsban vagyunk most
- melyik csúcsokban jártunk már

Részprobléma:

Hamiltonian Flights — Megoldás

Szerkezet: Az aktuális állapot két adattal írható le:

- melyik csúcsban vagyunk most
- melyik csúcsokban jártunk már

Részprobléma: `dp[u][s]` = az útvonalak száma, amelyek az 1-es várostól indulnak, az `u` városban végződnek és a városok `s` részhalmazát látogatták meg.

Kezdet: `dp[1][1]` = 1

Átmenet:

Hamiltonian Flights — Megoldás

Szerkezet: Az aktuális állapot két adattal írható le:

- melyik csúcsban vagyunk most
- melyik csúcsokban jártunk már

Részprobléma: $\mathtt{dp[u][s]}$ = az útvonalak száma, amelyek az 1-es várostól indulnak, az $\mathtt{u}$ városban végződnek és a városok $\mathtt{s}$ részhalmazát látogatták meg.

Kezdet: $\mathtt{dp[1][1]} = 1$

Átmenet: Legyen $\mathtt{v}$ a következő város, $\mathtt{u}$ a jelenlegi város. Ha van $\mathtt{u} \to \mathtt{v}$ járat és $\mathtt{v} \notin \mathtt{s}$:

$$\mathtt{dp[v][s \cup \{v\}]} += \mathtt{dp[u][s]}$$

Végső válasz: ans = $\mathtt{dp[n][\{1, 2, \ldots, n\}]}$

Ez az összes olyan útvonal száma, amely az 1-es várostól indul, a $\mathtt{n}$-edik városba ér, és minden várost pontosan egyszer érint.
Komplexitás: $\mathtt{O}(2^{\mathtt{n}} \cdot \mathtt{n}^2)$

Hamiltonian Flights — Kódvázlat

```
1  const int MOD = 1e9+7;
2  int n, m; cin >> n >> m;
3  vector<vector<int>> adj(n+1);
4  for (int i = 0; i < m; ++i) {
5      int a, b; cin >> a >> b;
6      adj[a].push_back(b);
7  }
8
9  vector<vector<long long>> dp(n+1, vector<long long>(1<<n, 0));
10 dp[1][1] = 1;
11
12 for (int mask = 1; mask < (1<<n); ++mask) {
13     for (int u = 1; u <= n; ++u) {
14         if (!(mask & (1<<(u-1)))) continue;
15         for (int v : adj[u]) {
16             if (mask & (1<<(v-1))) continue;
17             dp[v][mask | (1<<(v-1))] += dp[u][mask];
18             dp[v][mask | (1<<(v-1))] %= MOD;
19         }
20     }
21 }
```

Prime Multiples (CSES)

Adott k különböző prímszám: $p_1, \ldots, p_k$, és egy n szám. Hány olyan $1 \leq x \leq n$ egész szám van, amely osztható legalább az egyik p_i prímmel?

Bemenet: Első sor: n k. Második sor: p_1 p_2 $\ldots$ p_k.

Kimenet: Egy egész — azon $x \in \{1, \ldots, n\}$ számok száma, amelyek oszthatók legalább egy p_i-vel.

Korlátok: $1 \leq n \leq 10^{18}$, $1 \leq k \leq 20$.

Példa bemenet:

```
20 2
2 5
```

Kimenet:

```
12
```

Magyarázat: A 12 megfelelő szám: 2, 4, 5, 6, 8, 10, 12, 14, 15, 16, 18, 20.

Prime Multiples — Megoldás

Ötlet: Legyen $A_i = \{x \in [1, n] \mid p_i \mid x\}$, azaz az i-edik prím többszörösei.

A keresett számok száma:

$$|A_1 \cup A_2 \cup \cdots \cup A_k|$$

Példa: $n = 20$, $p_1 = 2$, $p_2 = 5$

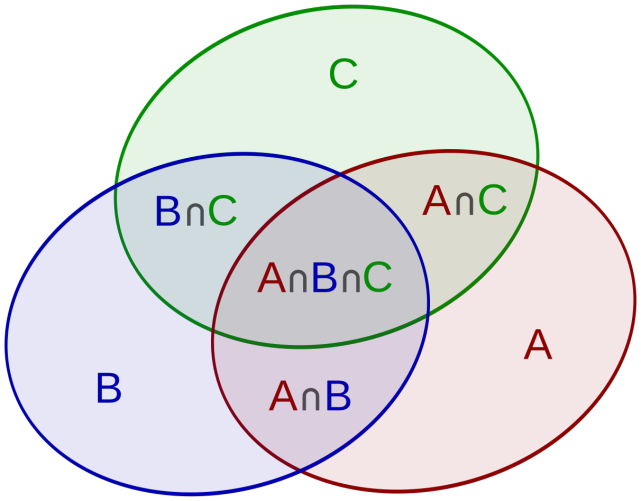
- $A_1 = \{2, 4, 6, 8, 10, 12, 14, 16, 18, 20\}$, $|A_1| = 10$
- $A_2 = \{5, 10, 15, 20\}$, $|A_2| = 4$
- $A_1 \cap A_2 = \{10, 20\}$, $|A_1 \cap A_2| = 2$

$$|A_1 \cup A_2| = |A_1| + |A_2| - |A_1 \cap A_2| = 10 + 4 - 2 = 12$$

Ha ennél több prím van?

Logikai szita - három halmaz

Legyen három halmazunk: A, B, C. Azt szeretnénk meghatározni, hány olyan x van, amely legalább az egyik halmazba tartozik: $|A \cup B \cup C|$.



A háromhalmazos eset képlete:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C| .$$

Logikai szita általánosan

Legyenek $A_1, A_2, \ldots, A_k$ halmazok. A cél: meghatározni, hogy hány x tartozik legalább egy halmazba: $\left|\bigcup_{i=1}^{k} A_i\right|$.

Logikai szita (inclusion–exclusion):

$$\left|\bigcup_{i=1}^{k} A_i\right| = \sum_{\emptyset \neq S \subseteq \{1,\ldots,k\}} (-1)^{|S|+1} \left|\bigcap_{i \in S} A_i\right| .$$

Magyarázat:

- S a halmazok egy nemüres részhalmazát jelöli, amelyeknek a metszetét vesszük: $\bigcap_{i \in S} A_i$ = azon elemek, amelyek minden $i \in S$-re A_i-nek elemei.
- Ha $|S| = 1$, akkor *hozzáadunk*.
- Ha $|S| = 2$, akkor *kivonunk*.
- Ha $|S| = 3$, akkor megint *hozzáadunk*.
- És így tovább, váltakozó előjellel.

Prime Multiples — Metszetek mérete

Legyen egy tetszőleges nemüres részhalmaz:

$$S \subseteq \{1, \ldots, k\}, \qquad S = \{i_1, i_2, \ldots, i_j\}.$$

A hozzá tartozó metszet:

$$\bigcap_{i \in S} A_i \quad = \quad \{x \in [1, n] \mid p_i \mid x \text{ minden } i \in S\}.$$

Mikor teljesül ez? Pontosan akkor, ha x osztható az összes S-ben levő prímmel, vagyis a szorzatukkal: $\mathrm{lcm}(p_{i_1}, \ldots, p_{i_j}) = p_{i_1} \cdot p_{i_2} \cdots p_{i_j}$

A metszet elemszáma:

$$\left| \bigcap_{i \in S} A_i \right| = \left\lfloor \frac{n}{\prod_{i \in S} p_i} \right\rfloor.$$

- 1 prím esetén: $\lfloor n/p_i \rfloor$
- 2 prím esetén: $\lfloor n/(p_i p_j) \rfloor$
- 3 prím esetén: $\lfloor n/(p_i p_j p_\ell) \rfloor$
- Általánosan: $\lfloor n/(\prod_{i \in S} p_i) \rfloor$

Prime Multiples — Bitmaszkos megvalósítás

A részhalmazokat programban kényelmesen bejárhatjuk bitmaszkkal: minden maszk ($1 \leq \text{mask} < 2^k$) egy $S \subseteq \{1, \ldots, k\}$ részhalmaznak felel meg.

Lépések:

- A maszk bitjei jelzik, mely prímek vannak S-ben.
- Kiszámoljuk a hozzátartozó szorzatot: $P = \prod_{i \in S} p_i$ és $\lfloor n/P \rfloor$ értékét.
- Előjel: $(-1)^{|S|+1}$.

Műveletigény: $O(2^k \cdot k)$, ami $k \leq 20$ esetén bőven belefér.

Prime Multiples — Kód

```
1  long long n, k; cin >> n >> k;
2  vector<long long> p(k);
3  for (int i = 0; i < k; ++i) cin >> p[i];
4  long long ans = 0;
5  int ALL = 1 << k;
6  for (int mask = 1; mask < ALL; ++mask) {
7      long long prod = 1;
8      bool too_big = false;
9      for (int i = 0; i < k; ++i) {
10         if (mask & (1 << i)) {
11             // jegyezzuk meg, ha tul nagy a szorzat
12             // meg a tulcsordulas elott
13             if (prod > n / p[i]) too_big = true;
14             prod *= p[i];
15         }
16     }
17     if (too_big) continue;
18     if (__builtin_popcount(mask) % 2 == 1) ans += n / prod;
19     else ans -= n / prod;
20 }
21 cout << ans << "\n";
```

Sum Over Subsets (SOS) — Általános probléma

Adott egy $\mathtt{A}$ tömb 2^n elemmel, ahol az indexek mindegyike egy $\mathtt{n}$ bites maszkot jelöl ($0, 1, \ldots, 2^n - 1$). Célunk: minden $\mathtt{x}$-re $(0 \leq \mathtt{x} < 2^n)$ kiszámítani:

$$\mathtt{F(x)} = \sum_{\mathtt{i} \subseteq \mathtt{x}} \mathtt{A[i]},$$

azaz az $\mathtt{A[i]}$ értékek összegét, ahol $\mathtt{i}$ bitmaszk **részhalmaza** $\mathtt{x}$-nek. Ez azt jelenti, hogy $\mathtt{i}$ pontosan azokkal a bitekkel rendelkezhet, amelyek $\mathtt{x}$-ben is szerepelnek.

Példa: Legyen $\mathtt{n} = 3$, így az indexek $0 \ldots 7$ közöttiek. Tekintsük az $\mathtt{x} = 5$ esetet (binárisan 101_2).

$$\mathtt{F(5)} = \mathtt{A[0]} + \mathtt{A[1]} + \mathtt{A[4]} + \mathtt{A[5]},$$

mivel ezek azok a maszkok, amelyek részhalmazai 101_2-nek:

$$0\ (000_2), \quad 1\ (001_2), \quad 4\ (100_2), \quad 5\ (101_2).$$

SOS — Részhalmazok felosztása

Cél: minden x-re kiszámolni

$$F(x) = \sum_{i \subseteq x} A[i].$$

Az ötlet: bontsuk fel az x-hez tartozó összes részhalmazt *rétegenként*, aszerint, hogy mely bitek egyeznek x-szel a magasabb pozíciókban.

Definíció: minden $0 \leq i \leq n$ egész számra:

$$S(x, i) = \{ y \subseteq x \mid (y \oplus x) < 2^i \}.$$

Azaz: $S(x, i)$ azoknak a x-beli részhalmazoknak a halmaza, amelyek az i-edik és alacsonyabb biteken **tetszőlegesen** eltérhetnek, de az i-nél magasabb biteken (a „felső” biteken) megegyeznek x-szel.

Fontos: $S(x, 0) = \{ y \subseteq x \mid y \oplus x < 1 \} = \{x\}$, és $S(x, n)$ tartalmazza x összes részhalmazát.

SOS - Példa a felosztásra

Legyen például $\mathtt{x} = 11010110_2$ (8 bites), és legyen $\mathtt{i} = 3$.

$$\mathtt{S}(11010110_2, 3) = \{\, \mathtt{y} \subseteq \mathtt{x} \mid \mathtt{y} \oplus \mathtt{x} < 2^3 \,\}.$$

Ez pontosan azokat a részhalmazokat jelöli, amelyekben a felső bitek (a 3-tól balra levő bitek) megegyeznek $\mathtt{x}$-szel. Tehát:

$$\mathtt{S}(11010110, 3) = \{11010000, 11010010, 11010100, 11010110\}.$$

(A felső 5 bit: 11010 mindegyikben egyezik; a legalacsonyabb 3 bit tetszőleges, de $\mathtt{x}$-nek részhalmaza.)

SOS - S(x,i) felbontás — rekurzív formula

Vizsgáljuk a következő lépést: hogyan kapjuk meg $\mathtt{S(x,\ i)}$-t az $\mathtt{i}-1$-es réteg ismeretében?

Ha az $\mathtt{i}$-edik bit $\mathtt{x}$-ben 0, akkor:

$$\mathtt{S(x,\ i)} = \mathtt{S(x,\ i-1)},$$

mert az $\mathtt{i}$-edik bitet nem lehet „kikapcsolni” (mivel $\mathtt{x}$-ben 0).

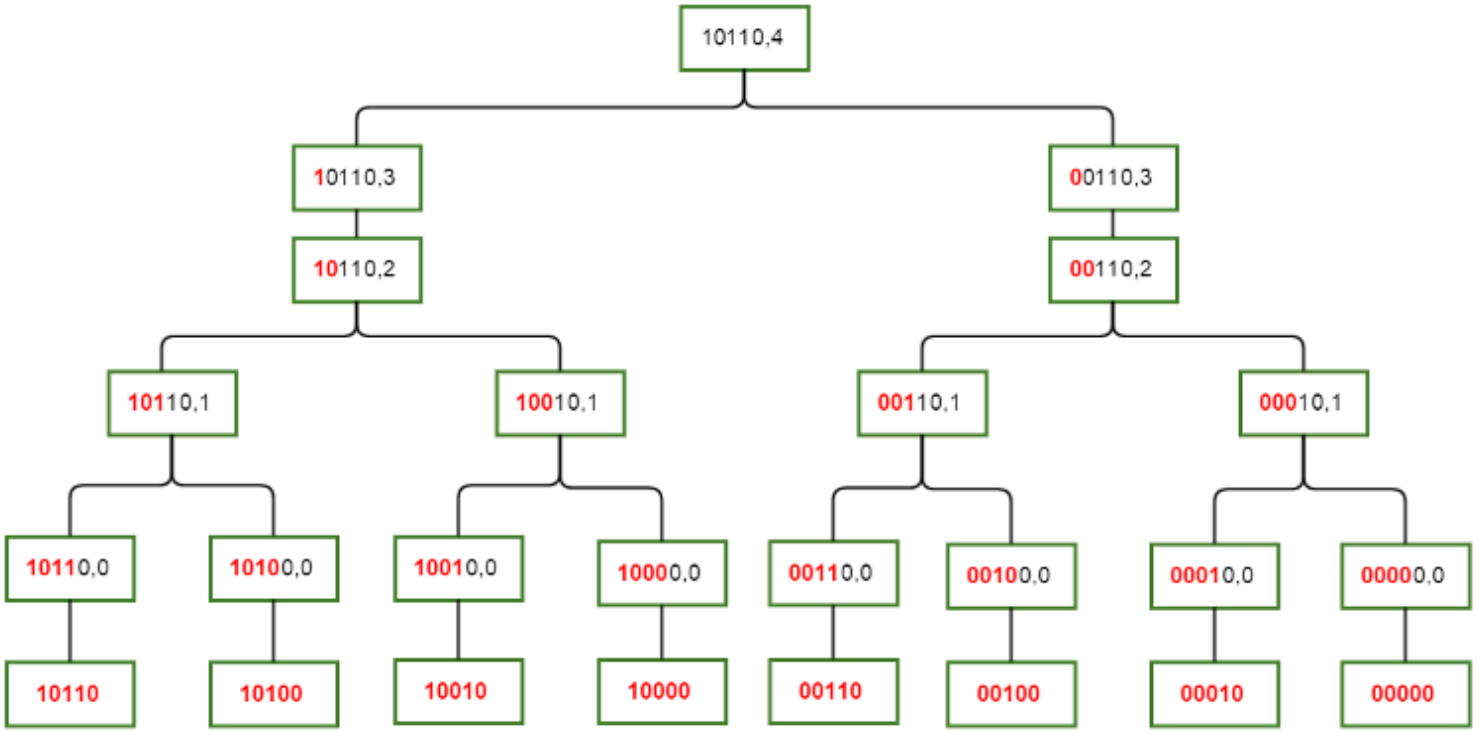
Ha az $\mathtt{i}$-edik bit 1, akkor az $\mathtt{S(x,\ i)}$ két részre esik:

- azok a részhalmazok, amelyekben az $\mathtt{i}$-edik bit is 1: ezek épp $\mathtt{S(x,\ i-1)}$,
- azok, amelyekben az $\mathtt{i}$-edik bit 0: ezek megegyeznek $\mathtt{S(x} \oplus 2^{\mathtt{i}}\mathtt{,\ i-1)}$-gyel.

Összefoglalva:

$$\mathtt{S(x,\ i)} = \begin{cases} \mathtt{S(x,\ i-1)}, & \text{ha az } \mathtt{i}\text{-edik bit } 0, \\ \mathtt{S(x,\ i-1)} \cup \mathtt{S(x} \oplus 2^{\mathtt{i}}\mathtt{,\ i-1)}, & \text{ha az } \mathtt{i}\text{-edik bit } 1. \end{cases}$$

SOS - S(x,i) felbontás — ábra



SOS - dp[x][i] definíció

A felbontás alapján vezessük be a következő segédfüggvényt:

$$\texttt{dp[x][i]} = \sum_{\texttt{y} \in \texttt{S(x,i)}} \texttt{A[y]}.$$

Megjegyzés:

- $\texttt{dp[x][0]} = \texttt{A[x]}$ (mivel $\texttt{S(x, 0)} = \{\texttt{x}\}$),
- a végső cél $\texttt{dp[x][n]} = \sum_{\texttt{y} \subseteq \texttt{x}} \texttt{A[y]} = \texttt{F(x)}.$

A rekurzió az $\texttt{S(x, i)}$-re látott rekurzív összefüggésből adódik:

$$\texttt{dp[x][i]} = \begin{cases} \texttt{dp[x][i-1]}, & \text{ha az } \texttt{i}\text{-edik bit } 0, \\ \texttt{dp[x][i-1]} + \texttt{dp[x} \oplus 2^{\texttt{i}}\texttt{][i-1]}, & \text{ha az } \texttt{i}\text{-edik bit } 1. \end{cases}$$

Ez egy dinamikus programozás, amely rétegenként (i szerint) bővíti az összegeket.

SOS - Kétdimenziós implementáció

Idő: $O(n \cdot 2^n)$. Memória: $O(n \cdot 2^n)$.

```
1  int n; // bitek száma
2  vector<int> a(1<<n); // eredeti tömb
3  vector<vector<long long>> dp(1<<n, vector<long long>(n+1,0));
4
5  for (int x = 0; x < (1<<n); ++x) dp[x][0] = a[x];
6
7  for (int i = 0; i < n; ++i) {
8      for (int x = 0; x < (1<<n); ++x) {
9          dp[x][i+1] = dp[x][i];
10         if (x & (1<<i)) {
11             dp[x][i+1] += dp[x ^ (1<<i)][i];
12         }
13     }
14 }
15
16 // eredmény: sos[x] = dp[x][n]
17 vector<long long> sos(1<<n);
18 for (int x = 0; x < (1<<n); ++x) sos[x] = dp[x][n];
```

SOS — Memóriaoptimalizált, leggyakrabb verzió

Megfigyelés: `dp[x][i+1]` csak `dp[·][i]`-tól függ, ezért az `i` dimenziót elhagyhatjuk.
Idő: $O(n \cdot 2^n)$, Memória: $O(2^n)$.

```
1  int n;
2  vector<long long> sos(1<<n); // bemenet másolata
3  for (int mask = 0; mask < (1<<n); ++mask) sos[mask] = a[mask];
4
5  for (int i = 0; i < n; ++i) {
6      for (int mask = 0; mask < (1<<n); ++mask) {
7          if (mask & (1<<i)) {
8              sos[mask] += sos[mask ^ (1<<i)];
9          }
10     }
11 }
```

Megjegyzések:

- A két ciklus sorrendje (először i, aztán mask) fontos: így az i-edik lépés hozzáadja azokat az tagokat, amelyeknek az i-edik bitje 1.
- A módszer általános: tetszőleges aggregáló műveletnél (összeg, min, max, XOR) alkalmazható, amennyiben a művelet asszociatív.

SOS Bit Problem (CSES) - Feladat

Adott egy n hosszúságú lista egész számokból: $x_1, \ldots, x_n$. Minden x_i elemre számold ki az alábbi három értéket:

1. a listában hány olyan y van, hogy $x \mid y = x$,
2. a listában hány olyan y van, hogy $x \,\&\, y = x$,
3. a listában hány olyan y van, hogy $x \,\&\, y \neq 0$.

Bemenet: Első sor: n ($1 \leq n \leq 2 \cdot 10^5$).
Második sor: n egész szám $x_1\ x_2\ \ldots\ x_n$ ($1 \leq x_i \leq 10^6$).

Kimenet: n sor — minden bemeneti elemhez egy sorban a három szám (a fenti sorrendben), szóközzel elválasztva.

Bemenet:

```
5
3 7 2 9 2
```

Kimenet:

```
3 2 5
4 1 5
2 4 4
1 1 3
2 4 4
```

SOS Bit Problem — Megoldás

Készítsünk frekvencia-tömböt:

$$\text{freq}[v] = \text{a bemenetben hány } x \text{ egyenlő } v\text{-vel.}$$

Minden bemeneti x esetén három érték kell:

1. $\#\{y : x \mid y = x\}$ — azaz hány y-nak vannak csak olyan bitei, amik x-ben is 1-ek ($\rightarrow y \subseteq x$). $\rightarrow$ Ez a **subset-sum**: $\sum_{s \subseteq x} \text{freq}[s]$.
2. $\#\{y : x \,\&\, y = x\}$ — azaz hány y-ban van benne minden x-beli 1-es bit ($\rightarrow y \supseteq x$). $\rightarrow$ Ez a **superset-sum**: $\sum_{t \supseteq x} \text{freq}[t]$.
3. $\#\{y : x \,\&\, y \neq 0\}$ — legalább egy közös 1-es bit: $N - \#\{y : x \,\&\, y = 0\}$.

   De $x \,\&\, y = 0 \Leftrightarrow y \subseteq (\overline{x})$. Tehát $N - \sum_{s \subseteq (\overline{x})} \text{freq}[s]$.

Mindhárom feladat SOS (Sum Over Subsets) variánssal oldható: subset-sum és superset-sum.

SOS Bit Problem — Kód

```
1  const int LOG = 20, M = 1 << LOG;
2  int N; cin >> N;
3  vector<int> A(N), freq(M);
4  for (int i = 0; i < N; i++) {
5      cin >> A[i];
6      freq[A[i]]++;
7  }
8
9  // subset-sum, superset-sum
10 vector<int> sub = freq, sup = freq;
11 for (int b = 0; b < LOG; b++)
12     for (int mask = 0; mask < M; mask++)
13         if (mask >> b & 1)
14             sub[mask] += sub[mask ^ (1 << b)];
15         else
16             sup[mask] += sup[mask ^ (1 << b)];
17
18 for (int x : A) {
19     int no_overlap = sub[(M - 1) ^ x];
20     cout << sub[x] << ' ' << sup[x] << ' ' << (N - no_overlap) << '\n'
21 }
```

Rekurzió memorizálással

(Memoization)

Rekurzió memorizálással

Probléma: Egyes esetekben nem könnyű meghatározni a részproblémák kiszámítási sorrendjét a DP megoldáshoz.

Ötlet: használjunk rekurziót, de ügyeljünk arra, hogy minden részprobléma megoldását csak egyszer számoljuk ki.

Megvalósítás: megjegyezzük a kiszámolt értékeket (memorizálás), és ha újra ki kellene számolni, akkor csak visszaadjuk a tárolt eredményt.

Másik alkalmazás: lehet, hogy nem a kiszámítási sorrend nehéz, csak éppen nem fér el a memóriában az összes DP érték. A rekurzió memorizálással során csak a tényleg szükséges értékeket számoljuk ki és tároljuk el.

Piramis — Feladat

Egy négyzet alapú piramist építenek. Az $\mathtt{N} \times \mathtt{N}$ méretű négyzetrács minden mezőjére adott az aktuális kockaszám $(\mathtt{A}_{\mathtt{i},\mathtt{j}})$. Egy követ a piramis széléről indítanak, és minden lépésben olyan oldalszomszédos mezőre lépnek, amelynek pontosan 1-gyel nagyobb a magassága. A követ ott helyezik el, ahol tovább már nem lehet lépni. Keressük a legtöbb lépésből álló ilyen útvonal hosszát, és adjuk meg az ehhez tartozó kezdőpozíciót (sor, oszlop).

Korlátok: $1 \leq \mathtt{N} \leq 1000$, $1 \leq \mathtt{A}_{\mathtt{i},\mathtt{j}} \leq 2\,000\,000$

Példa bemenet:

6

1	**2**	2	2	2	2
4	**3**	**4**	2	2	1
1	1	**5**	**6**	1	8
1	1	1	9	6	7
1	3	4	4	5	1
1	2	1	1	1	1

Kimenet:

5

1 1

Piramis — Megoldás (rekurzió memorizálással)

Megfigyelés: Ha egy (i,j)-ről induló optimális út első lépése (i',j'), akkor az (i',j')-ről induló folytatás is optimális a maradékra. (optimális részstruktúra)

Részprobléma:

`dp(i, j)` = az (i,j)-ről indulva megtett további lépések maximális száma.

(Tehát ha nem lehet tovább lépni, akkor `dp(i, j)` = 0.)

Rekurzió:

$$\texttt{dp(i, j)} = \max_{\substack{(i',j') \text{ szomszéd} \\ \texttt{A[i',j']=A[i,j]+1}}} \left(1 + \texttt{dp(i', j')}\right),$$

ellenkező esetben `dp(i, j)` = 0.

Kiszámítási sorrend? Macerás. Lehetne a pozíciókat az értékek szerint növekvő sorrendbe rendezni, de kényelmetlen. Helyette a rekurzív megoldásba építsünk memorizálást.

Megvalósítás: Rekurzió + memorizálás:

- minden cellára amikor először kiszámoljuk `dp(i, j)`-t akkor eltároljuk, és
- ha újra szükség van rá, visszaadjuk a tárolt értéket.

Piramis — Kód

```
1   #include <bits/stdc++.h>
2   using namespace std;
3   const int di[4] = {-1, 1, 0, 0}, dj[4] = {0, 0, -1, 1};
4   int n; vector<vector<int>> a, memo;
5
6   int dp(int i, int j) {
7       if (i < 1 || j < 1 || i > n || j > n) return -1;
8       if (memo[i][j] != -1) return memo[i][j];
9       int best = 0;
10      for (int d = 0; d < 4; d++) {
11          int ni = i + di[d], nj = j + dj[d];
12          if (a[ni][nj] == a[i][j] + 1) best = max(best, 1 + dp(ni, nj));
13      }
14      return memo[i][j] = best;
15  }
16
17  int main() {
18      cin >> n;
19      a.assign(n + 2, vector<int>(n + 2, -1));
20      memo.assign(n + 1, vector<int>(n + 1, -1));
21      for (int i = 1; i <= n; i++)
22          for (int j = 1; j <= n; j++) cin >> a[i][j];
23
24      int bestLen = 0, bi = 1, bj = 1;
25      for (int k = 1; k <= n; k++) {
26          vector<pair<int, int>> edges = {{1, k}, {n, k}, {k, 1}, {k, n}};
27          for (auto [i, j] : edges) {
28              if (dp(i, j) > bestLen) {
29                  bestLen = dp(i, j); bi = i; bj = j;
30              }
31          }
32      }
33      cout << (bestLen + 1) << "\n" << bi << "␣" << bj << "\n";
```

Longest Flight Route (CSES) - Feladat

Uolevi a lehető legtöbb várost szeretné bejárni egy repülőút során. Adott n város és m egyirányú járat. Határozd meg a Syrjälä-ból (1. város) Lehmälä-ba (n. város) vezető út maximális hosszát, valamint a városok sorrendjét.

Bemenet: Első sor: n m. Következő m sor: a b, ami azt jelenti, hogy van $a \to b$ járat.

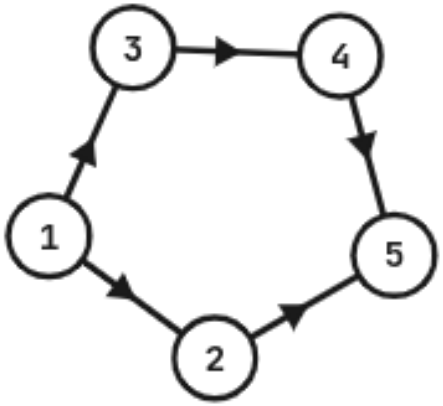
Kimenet: Első sor: az út hossza, vagy "IMPOSSIBLE", ha nincs út. Második sor: a városok sorszámai az útvonalon.

Példa bemenet:

```
5 5
1 2
2 5
1 3
3 4
4 5
```

Kimenet:

```
4
1 3 4 5
```



Longest Flight Route — Megoldás

Részprobléma:

$$\text{longest}(\mathtt{v}) = \text{leghosszabb út hossza } \mathtt{v} \to \mathtt{n} \text{ irányban}.$$

Alapeset:

$$\text{longest}(\mathtt{n}) = 1.$$

Rekurzív formula:

$$\text{longest}(\mathtt{v}) = \begin{cases} 0, & \text{ha } \mathtt{v}\text{-ből nem érhető el } \mathtt{n}, \\ 1 + \max\limits_{\mathtt{v} \to \mathtt{x}} \text{longest}(\mathtt{x}), & \text{egyébként}. \end{cases}$$

Memorizálás: Ha egyszer kiszámoljuk, eltároljuk, és legközelebb csak visszaadjuk az eredményt.

Előny: Nem kell a dinamikus programozás részproblémáinak a sorrendjét meghatározni (ami itt egy topologikus rendezést igényelne).

Longest Flight Route — A megoldás felépítése

Segédváltozók:

- `memo[v]` — a `longest(v)` értéke (kezdetben -1)
- `nxt[v]` — a következő város az optimális úton (rekonstrukcióhoz)

Rekurzív függvény:

```
1  int longest(int v) {
2      if (v == n) return 1;
3      if (memo[v] != -1) return memo[v];
4      memo[v] = 0;
5      for (int x : adj[v]) {
6          int len = longest(x);
7          if (len > 0 && len + 1 > memo[v]) {
8              memo[v] = len + 1;
9              nxt[v] = x;
10         }
11     }
12     return memo[v];
13 }
```

Rekonstrukció: Az út: $1 = v_1$, $v_{k+1} = \texttt{nxt}[v_k]$ amíg lehet.

Fibonacci Sequences - Feladat

Valerio egy sorozatot ír a táblára, aminek az első két eleme 1 és 2, majd mindig két szabály közül választva képzi a következő elemet:

- Az új szám a két előző összege.
- Az új szám a közvetlen előző kétszerese.

Davide látta az utolsó számot ($\mathtt{M}$), és szeretné tudni, hány különböző sorozatot írhatott Valerio.

Bemenet: Első sor: $\mathtt{T}$ - a tesztesetek száma ($1 \leq \mathtt{T} \leq 100\,000$).
Következő $\mathtt{T}$ sor: minden sorban egy egész szám: $\mathtt{M}$ ($2 \leq \mathtt{M} \leq 10^9$).

Kimenet: $\mathtt{T}$ sor, minden sor a lehetséges sorozatok száma az adott $\mathtt{M}$-re.

Példa bemenet:

```
2
8
39
```

Kimenet:

```
2
2
```

Magyarázat:
Pl. 8 esetén a két sorozat: 1, 2, 4, 8 és 1, 2, 3, 5, 8.
39 esetén: 1, 2, 3, 5, 8, 13, 26, 39 és 1, 2, 3, 6, 9, 15, 24, 39.

Fibonacci Sequences - Megoldás

Megfigyelés:

- Ha csak az összegzéses szabályt használjuk, akkor a Fibonacci-sorozatot kapjuk (1, 2, 3, 5, 8, 13, 21, ...).
- Ha egy x elemnél használjuk a kétszerező szabályt, és utána csak az összegzéses szabályt használjuk, akkor egy x-ből induló Fibonacci-sorozatot kapunk: x, $2x$, $3x$, $5x$, $8x$, ...

Jelölés: a Fibonacci-számok listája 2-től kezdve:

$$\text{fib} = [2, 3, 5, 8, \ldots]$$

Részproblémák: Legyen $f(x)$ a különböző sorozatok száma, amelyek vége pontosan x. (A megoldás $f(M)$).

Átmenet: $f(x)$ kiszámításához nézzük, hogy mely $1 \leq y < x$ számnál történt az utolsó kétszerezés (a legelején biztos történt, $y = 1$-re). A sorozat y-ig $f(y)$-féle lehet, onnantól kezdve pedig egyértelmű. Továbbá a fenti megfigyelés alapján szükségképpen létezik $\ell \in \text{fib}$, melyre $x = \ell \cdot y$. Tehát:

$$f(x) = \sum_{\substack{\ell \in \text{fib} \\ \ell \mid x}} f(x/\ell),$$

Alapeset: $f(1) = 1$ (a sorozat kezdete).

Fibonacci Sequences - Implementáció

Mivel `M` lehet 10^9 is, ezért dinamikus programozással nem számíthatjuk `f(x)`-et. Viszont ha rekurziót használunk memorizálással, akkor csak a tényleg szükséges értékeket számoljuk ki, így belefér a memória- és időlimitbe is a megoldás.

```
1   #include <bits/stdc++.h>
2   using namespace std;
3
4   vector<int> fib {2,3};
5   map<int,int> memo { {1, 1} };
6
7   int f(int x) {
8       if(memo.find(x) != memo.end()) return memo[x];
9       for(int i : fib)
10          if(x % i == 0)
11              memo[x] += f(x/i);
12      return memo[x];
13  }
14
15  int main() {
16      ios::sync_with_stdio(false); cin.tie(0);
17      while(fib.back() <= 1e9)
18          fib.push_back(fib.back() + fib[fib.size()-2]);
19      int T; cin >> T;
20      while(T--) {
21          int M; cin >> M;
22          cout << f(M) << endl;
23      }
24  }
```

Feladatok

Önálló munkára

Kitalálós játék - Feladat

`https://mester.inf.elte.hu` → Dinamikus programozás / 45.

Ádám és Éva kitalálós játékot játszik: Éva gondol egy számra 1 . . . `N`-ig, Ádámnak kérdéseket kell feltennie: "A gondolt szám kisebb vagy egyenlő-e, mint `x`?". A kérdéseknek ára van, minden `x`-re adott a kérdés `f(x)` ára. Határozd meg, hogy minimálisan mennyi pénz kell Ádámnak, hogy bármely számot ki tudja találni.

Bemenet:
Első sor: `N` $(1 \leq N \leq 500)$.
Második sor: `N` pozitív egész szám: `f(1), f(2), ..., f(N)` (≤ 1000).

Kimenet:
Egy egész szám: minimális költség.

Példa bemenet:

```
7
1 3 10 1 5 2 6
```

Kimenet:

```
14
```

Elevator Rides (CSES) - Feladat

https://cses.fi/problemset/task/1653

n ember szeretne feljutni egy épület tetejére, egyetlen lift van. Adott:

- $w_1, \ldots, w_n$ — az emberek súlya,
- x — a lift maximális terhelhetősége.

Határozd meg a minimális számú liftutat, amivel mindenki feljuthat.

Bemenet:
Első sor: n és x $(1 \leq n \leq 20,\ 1 \leq x \leq 10^9)$.
Második sor: $w_1\ w_2 \ldots w_n$ $(1 \leq w_i \leq x)$.

Kimenet:
Egy egész szám: a szükséges minimális liftutak száma.

Példa bemenet:

```
4 10
4 8 6 1
```

Kimenet:

```
2
```

Sailing Race (CEOI 2012) - Feladat

`https://oj.uz/problem/view/CEOI12_race`

Vitorlásversenyt szerveznek egy kerek tónál. $\mathtt{N}$ kikötő van számozva $1 \ldots \mathtt{N}$-ig a tó körül. A verseny több szakaszból áll, ahol egy szakasz egyenes vonal a kikötők között. Minden kikötőre adott, hogy onnan mely másik kikötőkbe lehet hajózni. Kétféle kérdés lehet: $\mathtt{k} = 0$ esetén nem lehet a szakaszoknak metszéspontja, $\mathtt{k} = 1$ esetén az *első* szakaszt metszheti legfeljebb egy másik szakasz. Maximálisan hány szakaszból állhat a verseny, és honnan induljon?

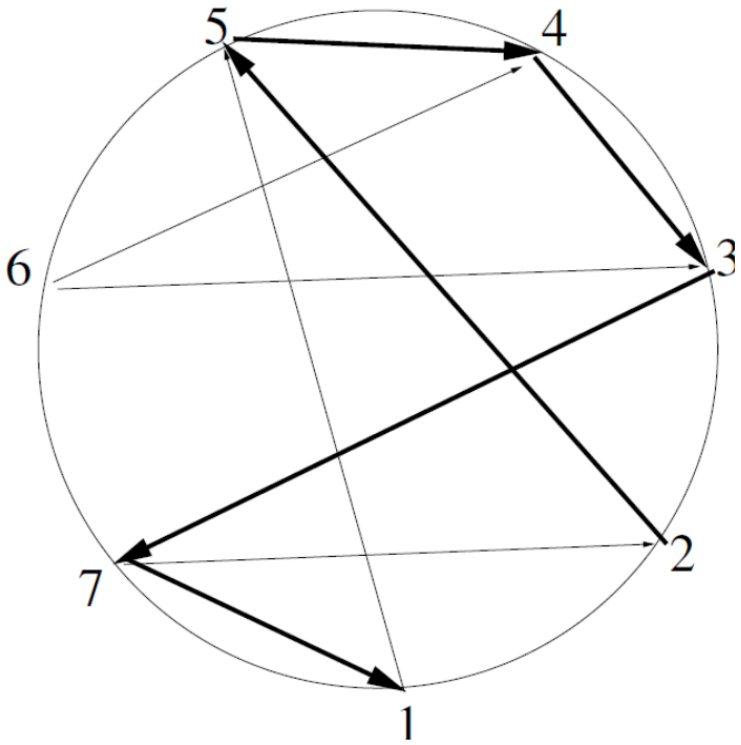
Bemenet:
Első sor: $\mathtt{N}$ és $\mathtt{k}$ $(1 \leq \mathtt{N} \leq 500,\ \mathtt{k} \in \{0, 1\})$.
Következő $\mathtt{N}$ sor: minden sor a kikötő közvetlen céljait tartalmazza, 0-val lezárva.

Kimenet:
Első sor: $\mathtt{M}$ — maximális szakaszok száma.
Második sor: a kezdő kikötő az egyik optimális pályához.

Sailing Race - Példa

Bemenet:

```
7 1
5 0
5 0
7 0
3 0
4 0
4 3 0
2 1 0
```



Kimenet:

```
5
2
```